GLOBALMART DATA ENGINEERING BOOTCAMP

Day 3 Individual SQL Assessment

Three SQL problems, your own logic, pen and paper only — plus one bonus round that has nothing to do with SQL at all.

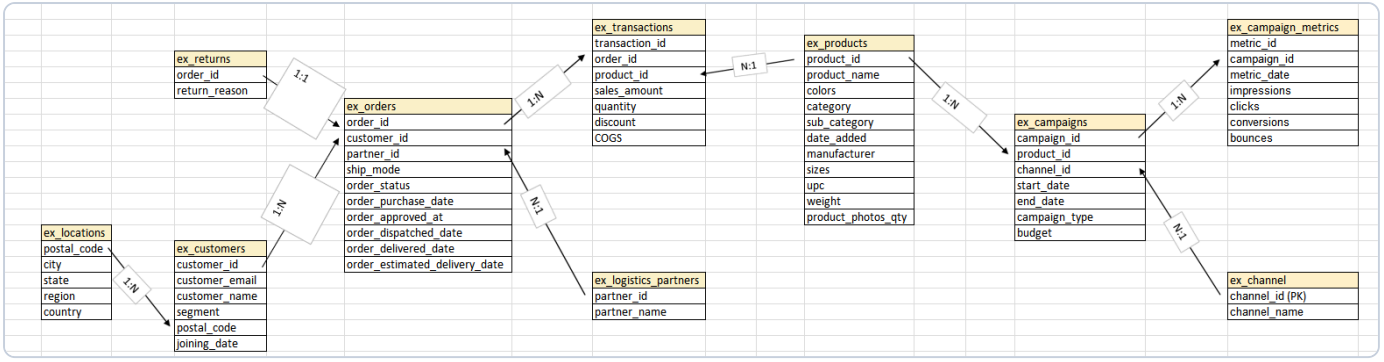
BEFORE YOU START

- No laptop, no phone, no internet. Just this booklet and a pen.
- Every learner's section opens with its own copy of the schema and syntax reference — flip back to it any time, you don't need to hold anything in memory.
- Write your SQL in the ruled space under each question.
- Hints are optional. Full marks either way.

If you get stuck, take a breath, re-read the hint, and remember: even Robert's "quick one" emails took three days to answer correctly.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Aayush Sharma

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Aayush Sharma, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Aayush suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Aayush Sharma, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Aayush as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Aayush Sharma, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Aayush Sharma's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Aayush sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

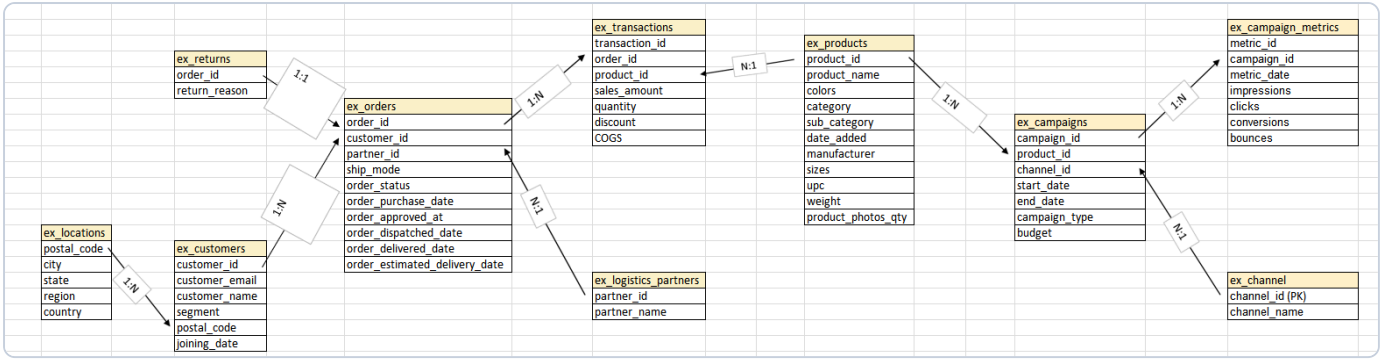
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Aditi

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Aditi, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Aditi suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Aditi, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Aditi as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Aditi, Finance sends the request this time — not Robert, a nice change of pace. *"We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week."*

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Aditi's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Aditi sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

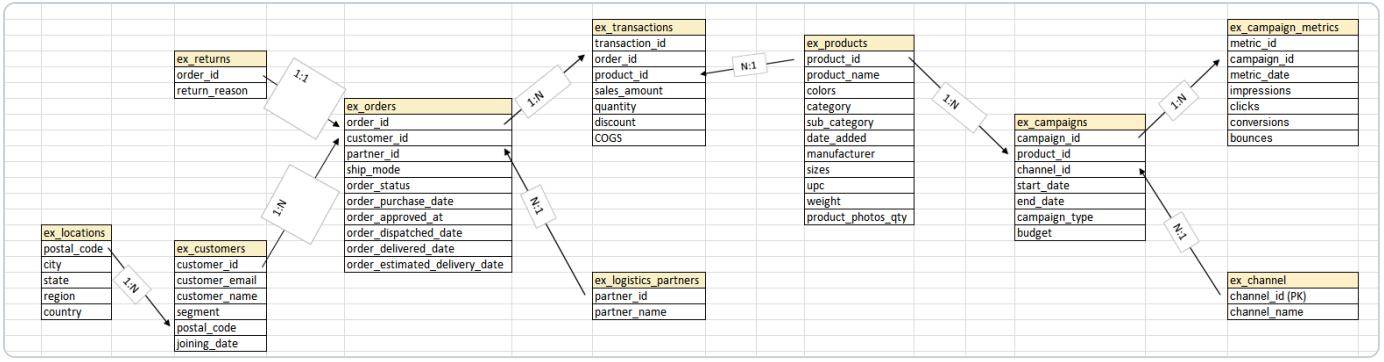
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Agnik Chakraborty

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Agnik Chakraborty, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Agnik suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Agnik Chakraborty, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Agnik as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Agnik Chakraborty, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Agnik Chakraborty's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Agnik sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

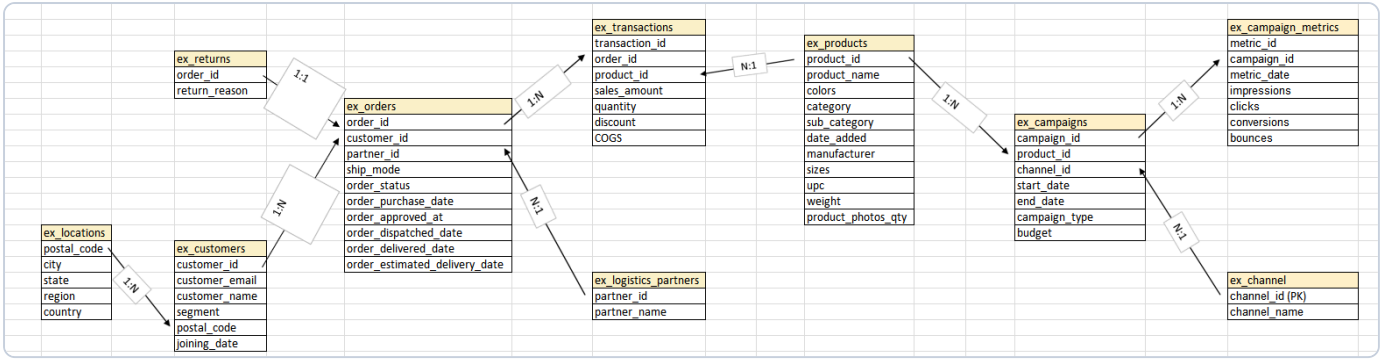
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Ananya Seth

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Ananya Seth, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Ananya suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Ananya Seth, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Ananya as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Ananya Seth, Finance sends the request this time — not Robert, a nice change of pace. *"We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week."*

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Ananya Seth's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Ananya sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

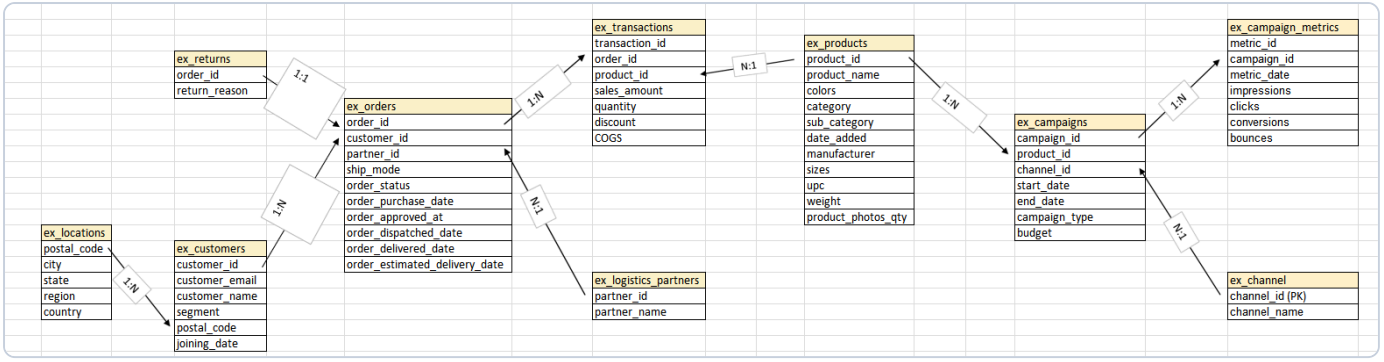
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Anuj Khokhar

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Anuj Khokhar, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Anuj suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Anuj Khokhar, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Anuj as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Anuj Khokhar, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Anuj Khokhar's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Anuj sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

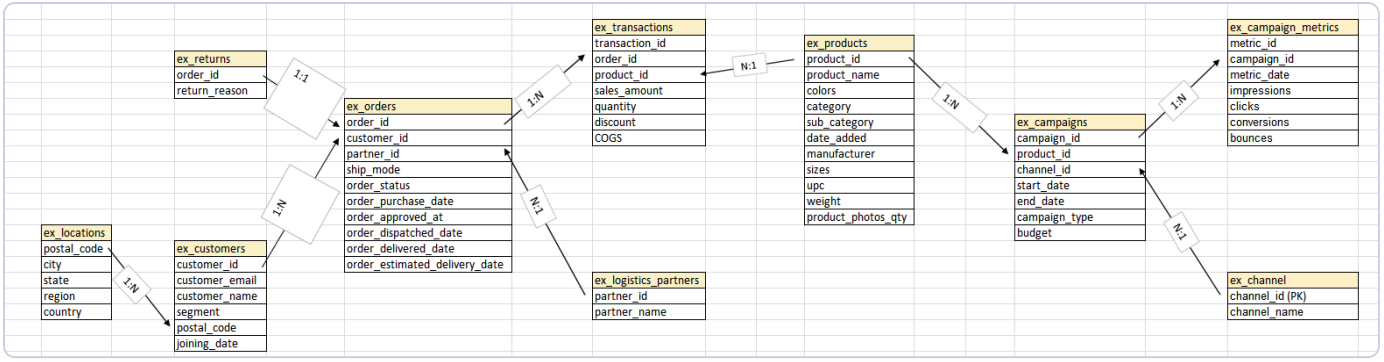
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Anuradha Navin

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Anuradha Navin, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Anuradha suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Anuradha Navin, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Anuradha as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Anuradha Navin, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Anuradha Navin's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Anuradha sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

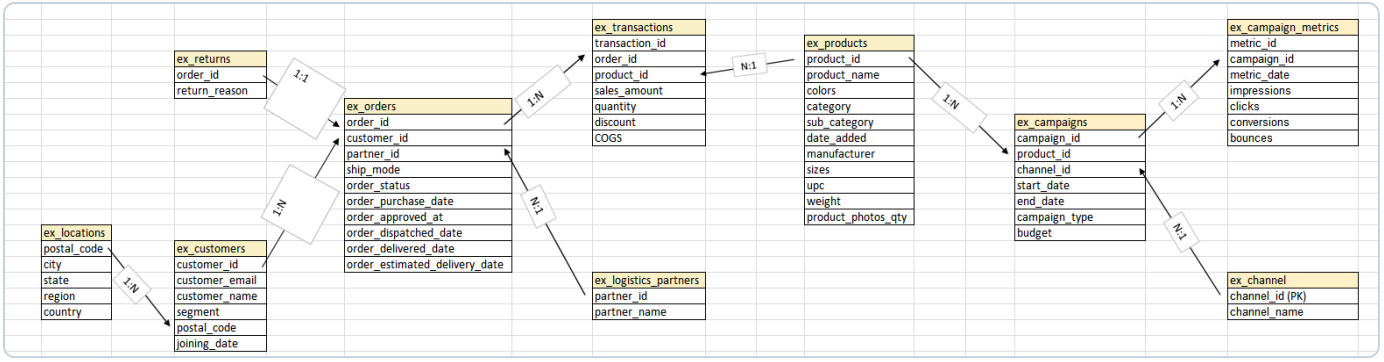
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Ayush Mishra

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Ayush Mishra, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Ayush suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Ayush Mishra, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Ayush as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Ayush Mishra, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Ayush Mishra's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Ayush sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

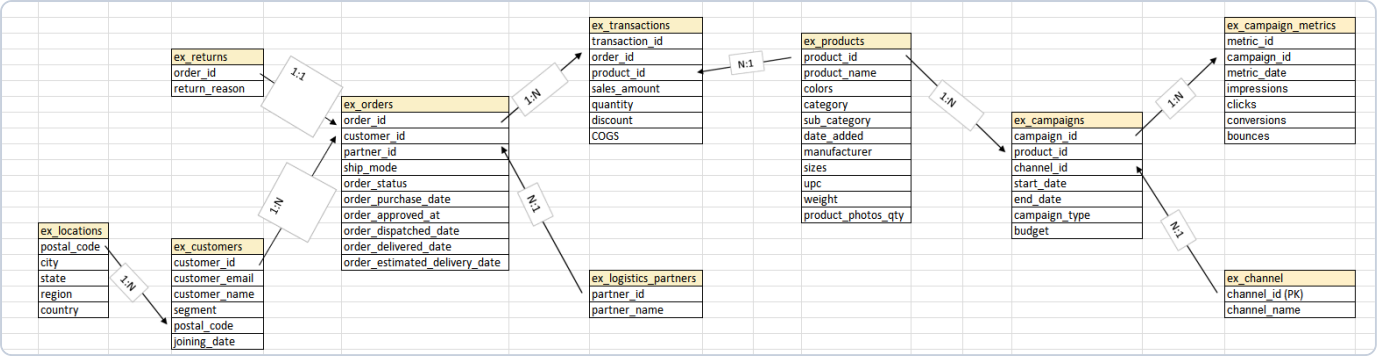
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Bhaskar Chauhan

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Bhaskar Chauhan, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Bhaskar suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Bhaskar Chauhan, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Bhaskar as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Bhaskar Chauhan, Finance sends the request this time — not Robert, a nice change of pace. *"We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week."*

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Bhaskar Chauhan's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Bhaskar sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

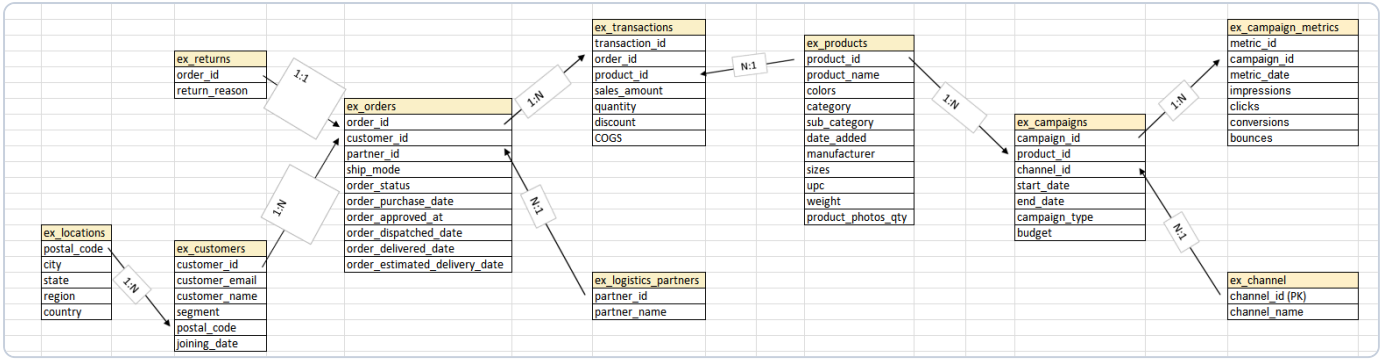
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Dev Jaishwal

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Dev Jaishwal, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Dev suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out

NOT GRADED — JUST FOR FUN

Dev Jaishwal, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Dev as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Dev Jaishwal, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Dev Jaishwal's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Dev sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

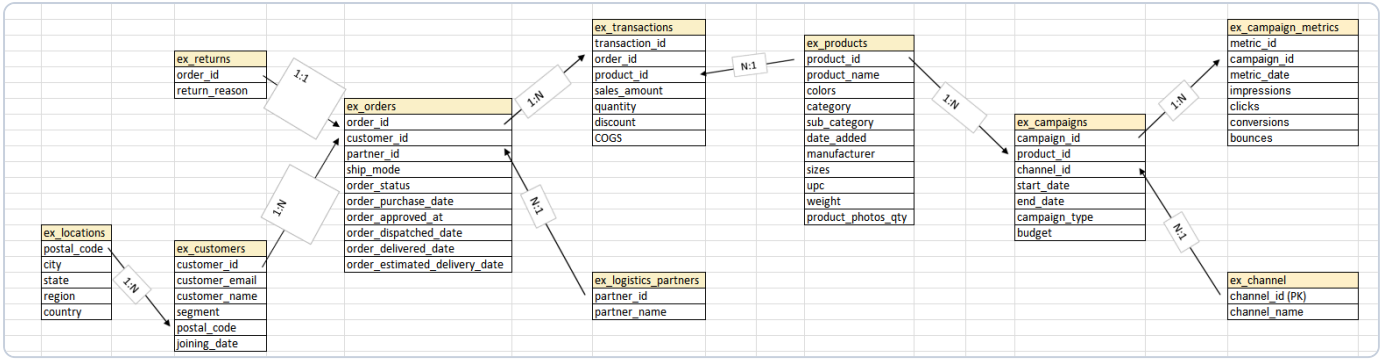
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Harsh Kumar

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Harsh Kumar, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Harsh suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Harsh Kumar, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Harsh as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Harsh Kumar, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Harsh Kumar's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Harsh sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

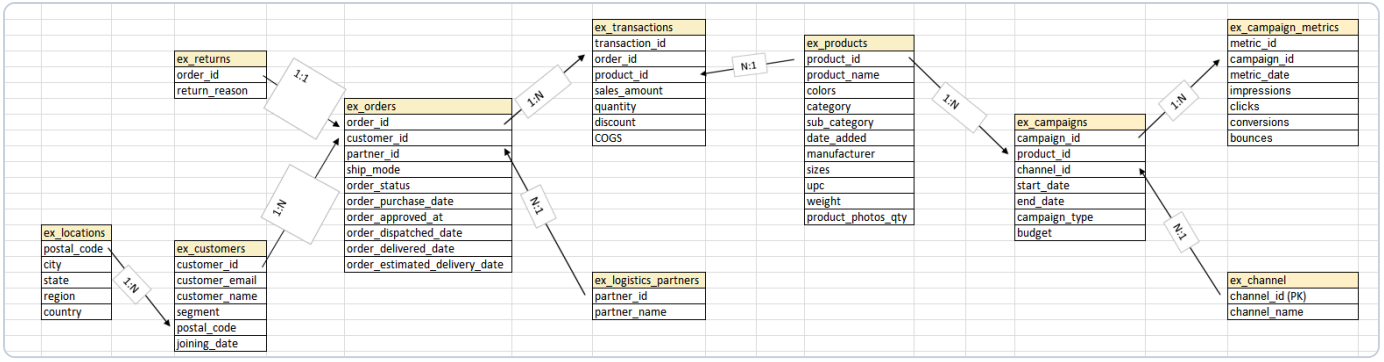
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Kabirkrishnan A

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Kabirkrishnan A, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Kabirkrishnan suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out

NOT GRADED — JUST FOR FUN

Kabirkrishnan A, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Kabirkrishnan as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Kabirkrishnan A, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Kabirkrishnan A's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Kabirkrishnan sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

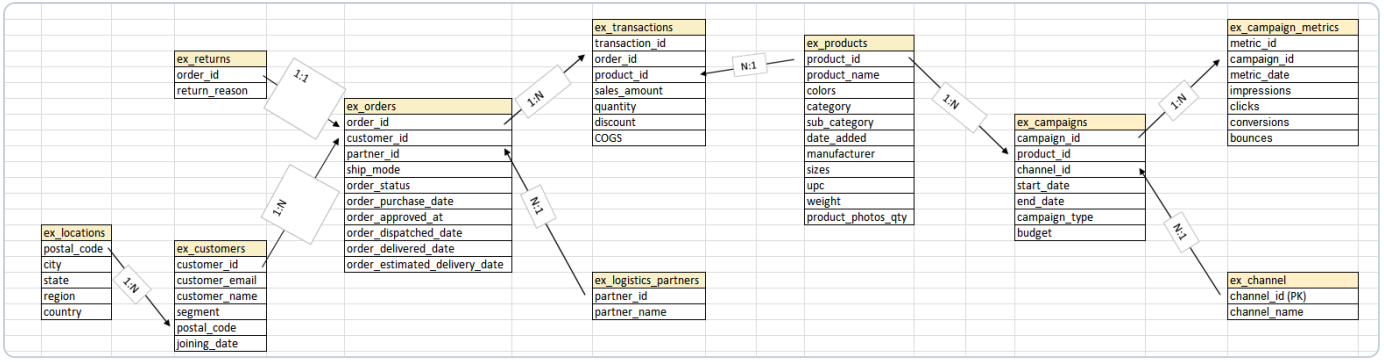
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Kriti Tiwari

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Kriti Tiwari, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Kriti suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Kriti Tiwari, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Kriti as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Kriti Tiwari, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Kriti Tiwari's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Kriti sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

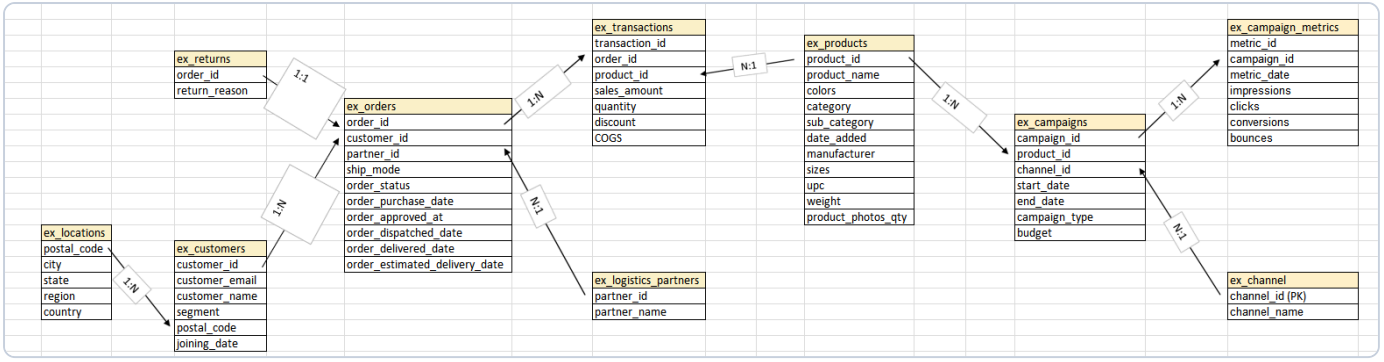
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Naman Prasad

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Naman Prasad, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Naman suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Naman Prasad, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Naman as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Naman Prasad, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Naman Prasad's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Naman sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

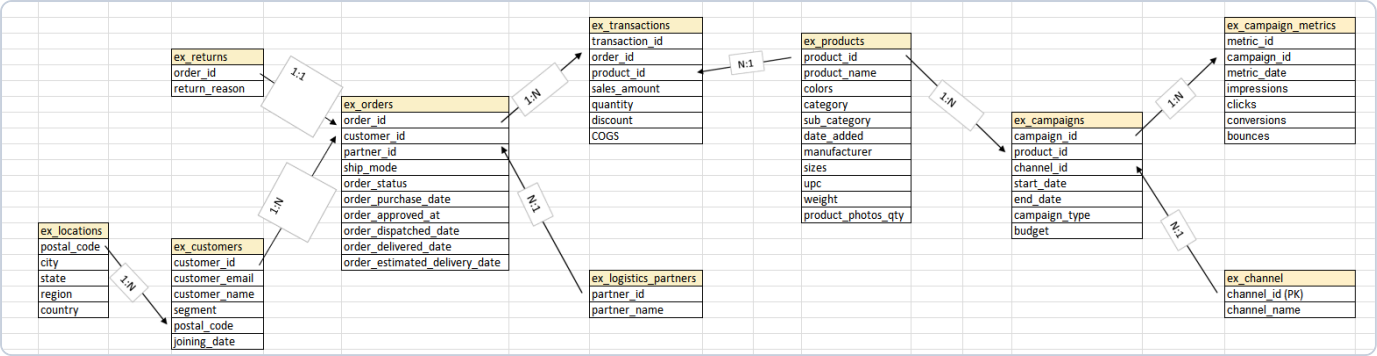
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Pratyush Tiwari

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Pratyush Tiwari, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Pratyush suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Pratyush Tiwari, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Pratyush as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Pratyush Tiwari, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Pratyush Tiwari's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Pratyush sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

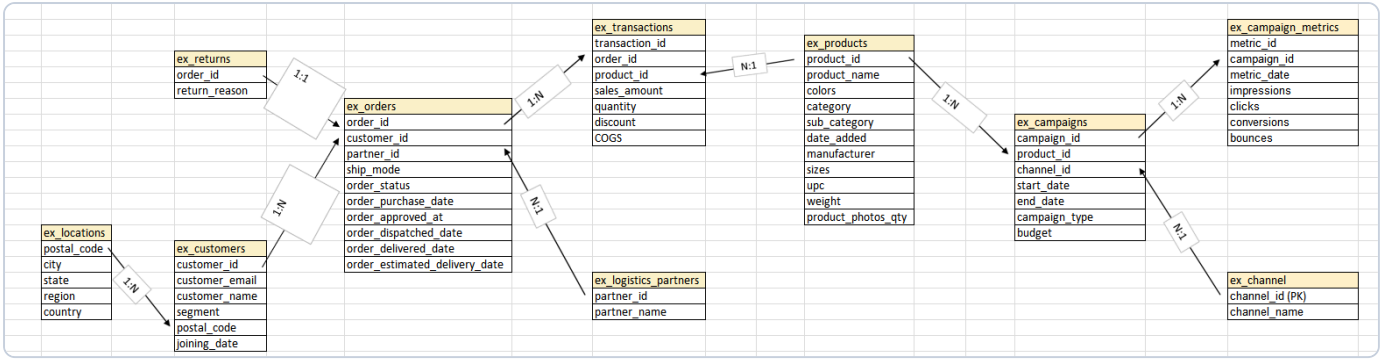
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Prem Kushwah

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Prem Kushwah, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Prem suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out

NOT GRADED — JUST FOR FUN

Prem Kushwah, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Prem as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Prem Kushwah, Finance sends the request this time — not Robert, a nice change of pace. *"We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week."*

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Prem Kushwah's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Prem sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

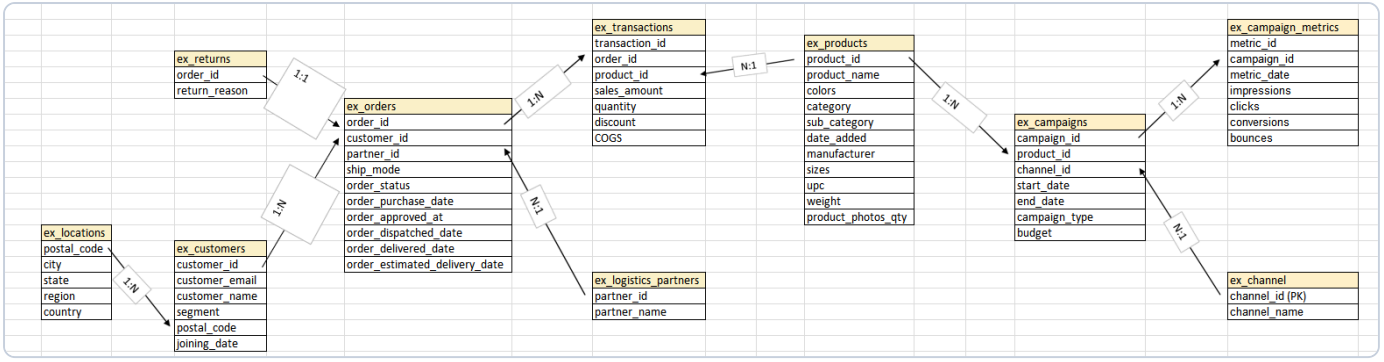
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Raghul D

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Raghul D, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Raghul suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out

NOT GRADED — JUST FOR FUN

Raghul D, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Raghul as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Raghul D, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Raghul D's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Raghul sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

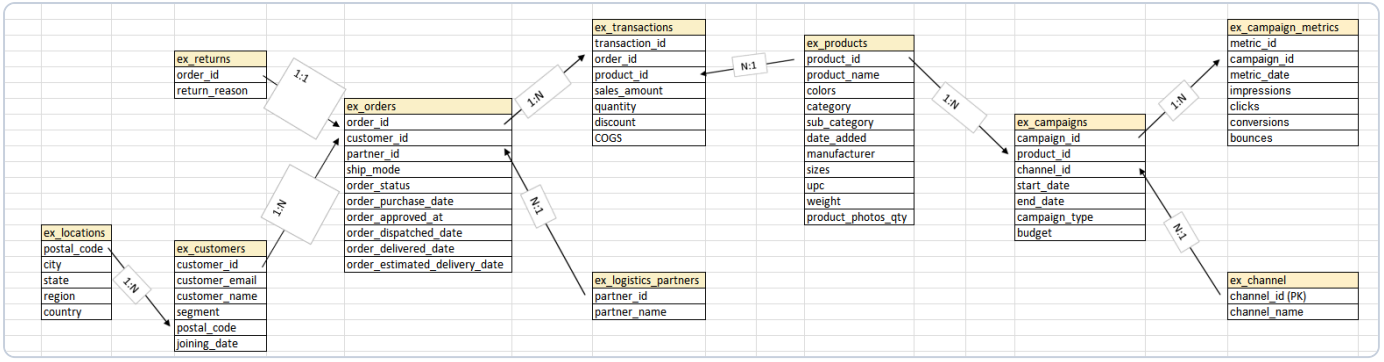
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Raghupatruni Thej

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Raghupatruni Thej, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Raghupatruni suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Raghupatruni Thej, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Raghupatruni as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Raghupatruni Thej, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Raghupatruni Thej's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Raghupatruni sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

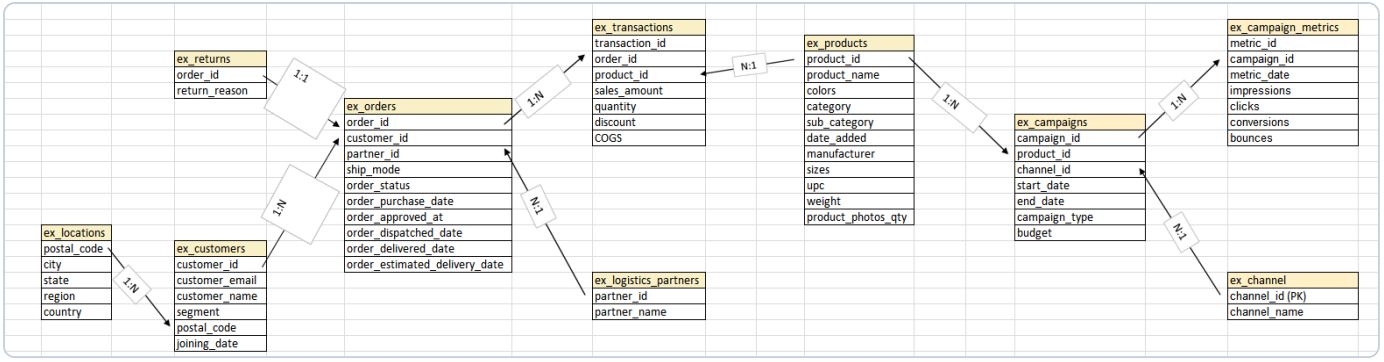
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Swarna Choudhury

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Swarna Choudhury, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Swarna suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out

NOT GRADED — JUST FOR FUN

Swarna Choudhury, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Swarna as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Swarna Choudhury, Finance sends the request this time — not Robert, a nice change of pace. *"We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week."*

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Swarna Choudhury's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Swarna sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

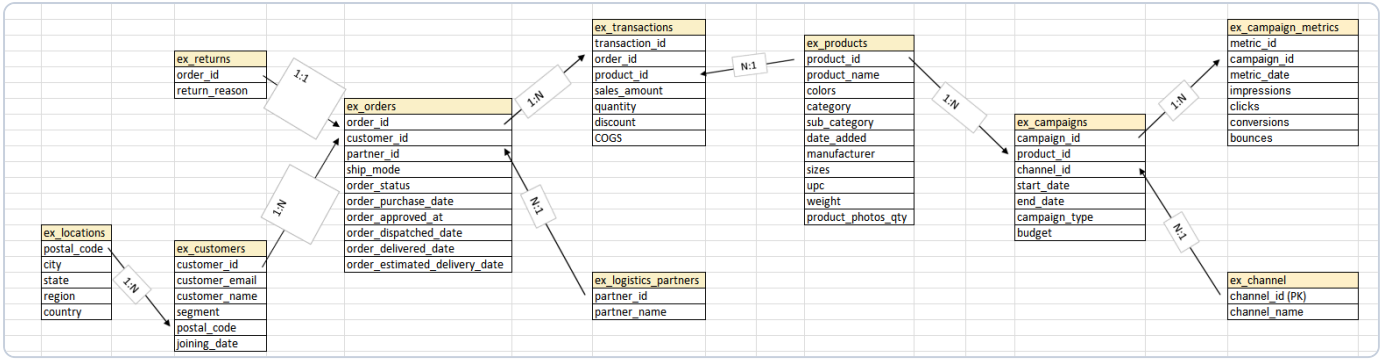
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Tanushi

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Tanushi, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Tanushi suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Tanushi, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Tanushi as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Tanushi, Finance sends the request this time — not Robert, a nice change of pace. *"We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week."*

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Tanushi's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Tanushi sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

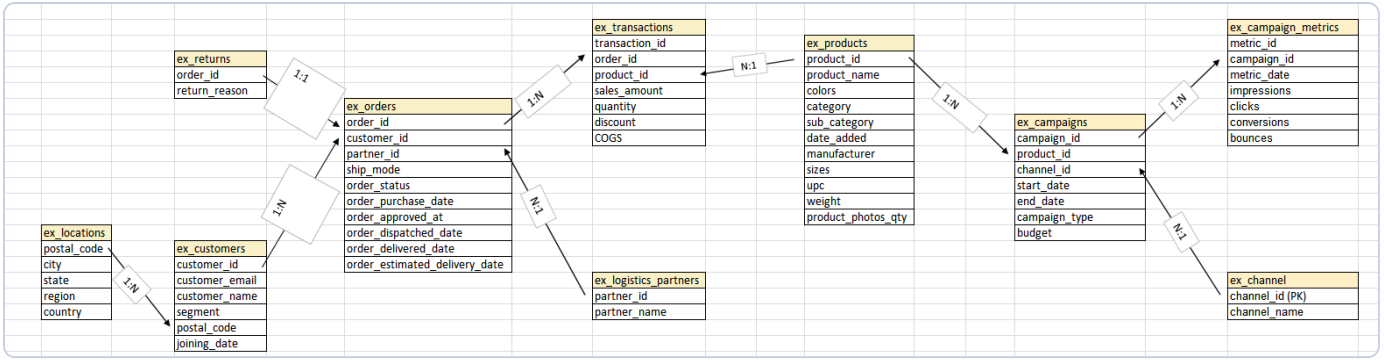
```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

Hint 1: Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

Hint 2: This query breaks more than one of those three at once — keep checking after you find the first issue.

REFERENCE — READ BEFORE ANSWERING

Schema (ER Diagram) & Syntax Cheat Sheet



All column names below are written exactly as shown in this diagram — including COGS in capitals, everything else lowercase.

JOIN skeleton

```
SELECT ...
FROM table_a a
JOIN table_b b ON a.key = b.key
JOIN table_c c ON b.key2 = c.key2
WHERE ...
GROUP BY ...
```

WHERE operators

```
=  <>  >  <  >=  <=
BETWEEN a AND b
IN (a, b, c)
LIKE 'A%'
IS NULL / IS NOT NULL
```

Aggregates + GROUP BY

```
SUM(col)  COUNT(col)  AVG(col)
COUNT(DISTINCT col)
GROUP BY col1, col2
```

Every non-aggregated SELECT column must be in GROUP BY.

Window function skeleton

```
RANK() OVER (
    PARTITION BY col
    ORDER BY other_col DESC
) AS rnk
```

Can't filter a window function in the same SELECT's WHERE — CTE it first, then filter.

Date functions

```
YEAR(date_col)
MONTH(date_col)
DATENAME(month, date_col)  -- "January"
DATEDIFF(day, d1, d2)
```

CASE WHEN

```
CASE WHEN cond THEN 'A'
     WHEN cond2 THEN 'B'
     ELSE 'C' END
```

PIVOT skeleton

```
SELECT key_col, [2017], [2018]
FROM long_format_table
PIVOT(
    SUM(value_col)
    FOR [year_col] IN ([2017],[2018])
) AS pvt
```

Square brackets around pivoted values are mandatory.

Optimization checklist (Day 3)

```
1. No SELECT *
2. Filter with WHERE, not HAVING
3. FROM the biggest table first,
   JOIN outward to smaller ones
4. GROUP BY beats DISTINCT
```

DAY 3 INDIVIDUAL ASSESSMENT

Udhav Vinaik

3 SQL problems + 1 for-fun bonus · pen and paper · hints available below each question

Q1 The Logistics Partner Scorecard

Udhav Vinaik, Operations forwards a message from the VP: "*Before we renew any shipping partner contracts next quarter, I want a real scorecard — not vibes. Who is actually fast, and who is quietly costing us customers?*" Nobody on the team has ever actually pulled this number before, which Udhav suspects is either a very good sign or a very bad one.

Task: for each logistics partner, calculate how many delivered orders they have handled and their average number of days from purchase to delivery. Rank the partners from fastest average delivery time (rank 1) to slowest, so the VP knows exactly who to keep and who to have an uncomfortable conversation with. Only count orders that actually reached the customer — an order still "in transit" doesn't have a delivery time to average yet.

```
Expected columns: partner_name | delivered_order_count | avg_delivery_days | speed_rank
```

Hint 1: Join ex_orders to ex_logistics_partners on partner_id. Filter to order_status = 'delivered' first — you can't average a delivery time that hasn't happened.

Hint 2: avg_delivery_days = AVG(DATEDIFF(day, order_purchase_date, order_delivered_date)), grouped by partner. Rank with RANK() OVER (ORDER BY avg_delivery_days **ASC**) — ascending, because here lower is better, unlike most ranking examples you've seen so far.

Q4 Draw It Out NOT GRADED — JUST FOR FUN

Udhav Vinaik, put the pen down on SQL for a minute. Three cases cracked today: the Logistics Partner mystery, the Segment Profit mystery, and the Slow Query crime scene. In the box below, draw Udhav as the detective who solved all three — stick figures welcome, magnifying glass optional, artistic talent 100% not required. This is the one question on the entire paper where "I have no idea" is a completely acceptable technical approach.

Q2 Profit by Segment, Year over Year

Udhav Vinaik, Finance sends the request this time — not Robert, a nice change of pace. "*We want to know if our Home Office customers are becoming more profitable, or if we're just moving more units to them at thinner and thinner margins. Same question for Consumer and Corporate. Show me 2017 versus 2018, side by side, the way you laid out that order-count report earlier this week.*"

Task: for each customer segment, calculate total profit (sales amount minus cost of goods sold) for 2017 in one column and 2018 in another — the same pivoting technique from the PIVOT session, just pointed at a completely different question this time.

```
Expected columns: segment | [2017] | [2018]
```

> **Hint 1:** Build the long version first: one row per segment per year with SUM(sales_amount - COGS) as profit — join ex_transactions → ex_orders → ex_customers, and use YEAR(order_purchase_date).

> **Hint 2:** Same PIVOT skeleton as the reference sheet, just pivoting a profit total by segment instead of an order count by month: `FOR order_year IN ([2017],[2018])`.

Q3 Spot the Slow Query

Udhav Vinaik's manager forwards a query with the subject line "this works fine, not sure why everyone's complaining." It was written by a contractor who left the project three weeks ago — conveniently, right before the optimization session Udhav sat through on Day 3. The query below does return the correct numbers; Finance has been using its output all month without one complaint about accuracy. The complaint is entirely about the 4 minutes and 40 seconds it takes to run, every single time someone refreshes the dashboard.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
  AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

Task: (a) list 3 specific things wrong with this query — "it's slow" doesn't count as one of them — and (b) rewrite it so it does the same job properly.

```
Expected columns: Written answer: 3 named issues + 1 rewritten query
```

> **Hint 1:** Re-read Best Practices 1–3 from the optimization session: don't SELECT *, filter with WHERE not HAVING, and start FROM the table with the most rows instead of the smallest one.

> **Hint 2:** This query breaks more than one of those three at once — keep checking after you find the first issue.